

Exercise 1 (General Questions : 5 points) :

For the following questions, a justification must be always provided, Yes/No answers will **not be** accepted unless a convincing explanation is given.

(We ended up relaxing even the justification for most questions unless you provide a justification which is totally out of context)

1. Is the language of all possible words composed over $\{a,b\}$ regular?

a. True

b. Justification : Because there is a regex $(a|b)^$ or we draw the DFA for it. All words, this includes the empty string which is a word too. We are not speaking here about the English words ?*

2. The epsilon transition (ϵ -transition) is allowed in Deterministic Finite Automata (DFA). True or False? .

a. False

b. As soon as we create an epsilon transition, the DFA becomes non-deterministic as we will always have an extra choice to jump to another state without the need to read.

3. This regular expression, $1^*|(1^*01^*01^*01^*)^*$, can be used to match binary strings that contain a number of zeros divisible by 3. True or False?

a. False

b. 000 can not be generated by this regular expression. Because, We are forcing the string to always have a ONE, This is the RegEx printed to you : $1^|(1^*01^*01^*01^*)^*$ NOT $1^*|(1^*01^*01^*01^*)^*$*

4. The language $L=\{wxzxw^R \mid w, x, z \in \{0,1\}^*, |w|>0, |x|>0 \text{ and } w^R \text{ is the reverse of } w\}$. L is **not** a regular language. True or False?

a. True , it is NOT Regular

*b. Even through, it may seem that We can create the regex : $11(0|1)^*11 \mid 10(0|1)^*01 \mid 00(0|1)^*00 \mid 01(0|1)^*10$.*

There are other words that are in the language but we cannot model them : $1 \ 1110 \ 1110 \ 1$ as the x here can take two, three, four ... letters.

5. Is the language $L=\{a^n a^n \mid n \text{ is a natural number}\}$ a regular language ? True or False?

a. True

b. The language of even-length words composed of a. Regular Expression $(aa)^$*

Exercise 2 (Regular Languages : 5 points) :

1. [2 points] Provide the regular expression for languages:

a. Where all words in which each pair of 0s (i.e. **00**) is followed by a pair of 1s

$(1|01|0011)^*(0|\epsilon)$ (Counter Example being used 010 + others)

b. Whose words do not contain the substring 100

$0^*(1|10)^*$ (Counter Example being used 010 + others)

2. [1 point] Using the pumping lemma or fooling set, prove that the language $L=\{a^n b a^n \mid n \text{ is a natural number}\}$ is **not regular**.

We assume that the language is regular . . .

If p is pumping length, the string $S=xyz=a^p b a^p$ which is in the language can be pumped to generate more words that belong to the language.

because of the pumping lemma condition that $|xy| \leq p$

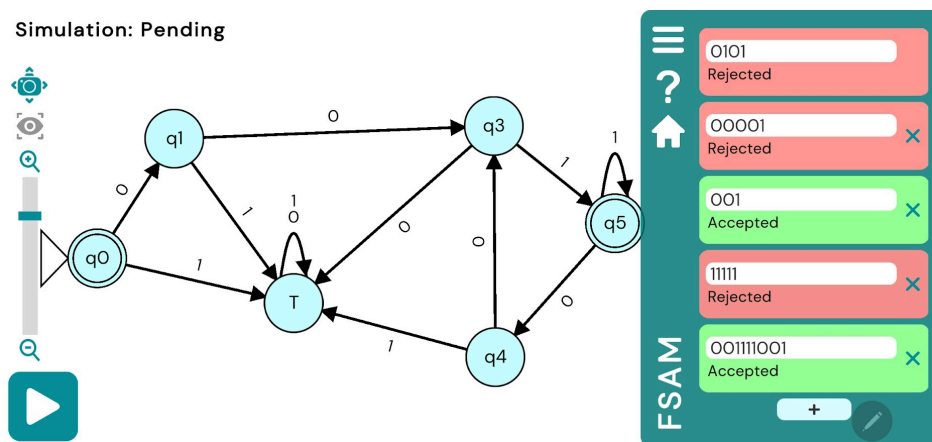
It implies that xy contains only a from the left side of b

by pumping y , we will have more a s to the left of b ($p+k$) such that $k=|y|$

...

3. [3 point] Draw strictly the DFAs for the following languages :

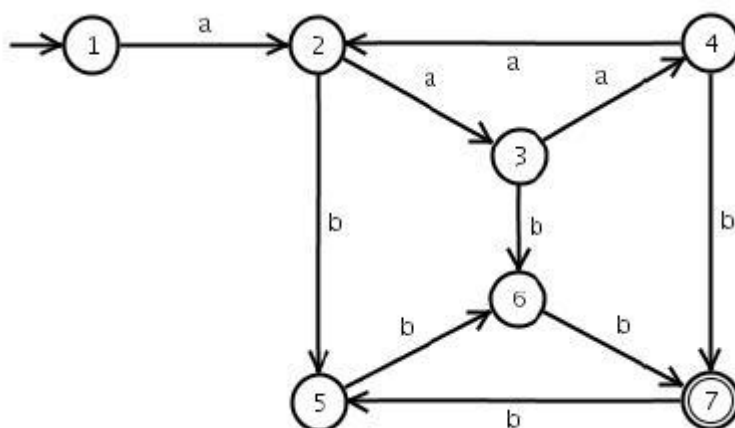
a. $(001+)^*$ (+ means 1 or more) :



b. $L = \{ a^k b^m \mid k+m = 3n + 1 . k, m, n \in \{1, 2, 3, \dots\} \}$ over the alphabet $\{a,b\}$.

- Missing transitions must point to a Trap state (Example 1 $\rightarrow b \rightarrow T \dots$)

- For the case when $n=0$, we relaxed this during the marking.



Exercise 3 (Context-Free Languages : 5 points) :

1. [2 point - **Optional Exerc**] Create the grammar for the languages
- a. $L = \{ a^i b^j c^k \text{ such that } j = i + k \} \text{ over } \{a,b,c\}$ (number of b=number of a+c)

$$S_0 \rightarrow aS_1bS_2 \mid S_1bS_2c \mid \varepsilon$$

$$S_1 \rightarrow aS_1b \mid \varepsilon$$

$$S_2 \rightarrow bS_2c \mid \varepsilon$$

- b. $L = \{ w\#x \mid w^R \text{ is a substring of } x \text{ for } w, x \in \{a,b\}^* \}$

$$S \rightarrow CB$$

$$C \rightarrow 0C0 \mid 1C1 \mid \#B$$

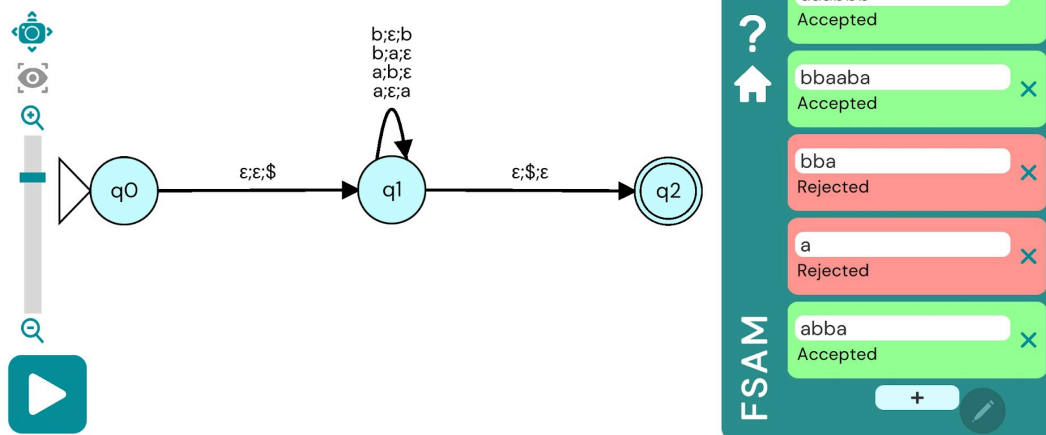
$$B \rightarrow AB \mid \varepsilon$$

$$A \rightarrow 0 \mid 1$$

2. [1 point - **Optional Ex**] Create the PDA for $L = \{ w \in \{a, b\}^* : w \text{ has equal numbers of a's and b's} \}$.

We accepted solutions who converted a CFG to PDA

Simulation: Pending



3. [1 point] Transform the following grammar into Chomsky Normal Form (CNF):
- We look mostly for the form of the final grammars even if there are mistakes, we accept.*

$$S \rightarrow aSSA \mid ASC \mid a$$

$$A \rightarrow AAB \mid B \mid C$$

$$B \rightarrow a \mid \varepsilon$$

$$C \rightarrow AC \mid CB$$

$$S \Rightarrow T_a X_1 \mid T_a X_3 \mid a$$

$$A \Rightarrow A X_4 \mid AB \mid a$$

$$B \Rightarrow a$$

$$T_a \Rightarrow a$$

$$X_1 \Rightarrow S X_2$$

$$X_2 \Rightarrow SA$$

$$X_3 \Rightarrow SS$$

$$X_4 \Rightarrow AB$$

4. [2 point, **Given in lecture**] Prove using the pumping lemma that $L = \{ 0^n 1^m \mid n \text{ and } m \text{ are natural numbers} \}$ such that m is a multiple of n . 0^n means 0 being repeated n times. **As opposed to all questions, This exercise is strictly marked, any small error, zero mark is given directly.**

*The principle : From a word S , you can find a **single** placement for vxy to reach/generate any word using one/two pumps (being pumped at the same time i.e. same i for v and y incremented at the same time).*

We assume that $S = 0^p 1^p$ in the language such that P is the pumping length ...

S can be written as $uvxyz$ such that $|vxy| \leq P$ and $|vy| > 0$

Case 1 : vxy in the zero part, we pump up, the number of zeros would be greater than the number of 1. Therefore, it is impossible for the number of 1s to be a multiple of zeros. We will not be able to generate words in the language from this position.

Case 2 : vxy on the one part (If you say, I pump down, it is totally incorrect). You can of course generate words in the language in the form : $0^p 1^{pk}$ when $|vxy| = P$ as long as the number of zeros is fixed at P .

BUT, we would never be able to generate words where the number of 0 is $P+1$ or $P+2$,,, because vxy is on the side of the 1s.

Case 3 : v is on the side of zeros and y on the side of 1s:

When we pump up/down, we MUST do them both at the same time. Therefore if we consider $|v| = H$ and $|y| = C$,

We can only generate :

$0^{P+H} 1^{P+C}$
 $0^{P+2H} 1^{P+2C}$
 $0^{P+3H} 1^{P+3C}$
 ...

Let's assume that $P+C$ is a multiple of $P+H$

But, we can never generate words in the form : $0^{P+H} 1^{P+C+(P+H)}$, $0^{P+H} 1^{P+C+(P+H)+(P+H)} \dots$

As we pump at the same time v and y so therefore, increase the number of zeros and ones at the same time and move on to the next iteration of pumping. We don't have the option where we pump v or y separately.

(Of course, you can struggle to prove that $P+C$ can not be a multiple of $P+H$... but C can be equal to H ... As unknowns can be anything..)

Case 4 : v or y contains a mixture of 0 and 1 : is not possible, because if we pump 00111 and we will change the strict order being imposed, 0 followed by $1 \rightarrow 001111001111\dots$

Exercise 4 (Turing Machines : 3 points) :

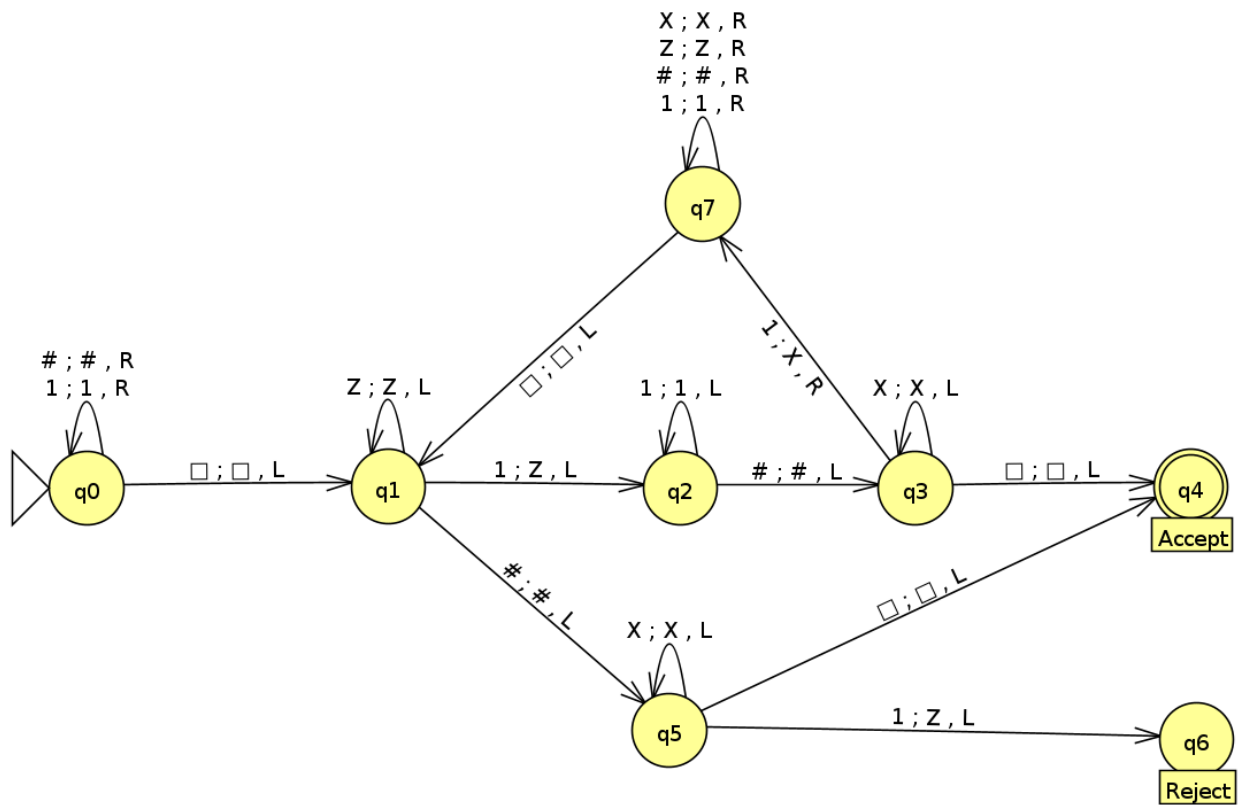
Construct the turing machines for the following languages :

- [1 points] Construct a decider (NOT Transducer) turing machine (1point) to decide on whether two unary numbers separated by # are sorted such that the small number is written first (on the left side of the #). The alphabet used is only $\{1\}$.

For instance :

$111\#1$ should be **rejected**. The first number has more digits/characters.
 $1\#1111$ should be **accepted**. The first number has less digits/characters.
 $11\#11$ should be **accepted**. Both numbers have the same number of characters.

Don't bother with cases where a number is not given ($111\#$), the numbers are strictly given to you . No need to return the tape head the beginning.



2. [2 points] Write the algorithm and construct the decider Turing machine that decides on:

$$A = \{0^{2^n} \mid n \geq 0\}$$

The language consists of all strings of 0s whose length is a power of 2.

Examples :

00 (n=1, 2 zeros)
 0000 (n=2, 4 zeros)
 00000000 (n=3, 8 zeros)
 0000000000000000 (n=4, 16 zeros)

